DEPLOYMENT DOCUMENTATION

Predictive Maintenance and Failure Risk Assessment for Industrial Equipment

SANIKOMMU DINESH REDDY
S20240010215
UG-3 CSE

Production Cloud Deployment Reference & Verification Guide (Render Hosting)

1. Project & Deployment Overview

This document details the production cloud deployment configuration, operational architecture, verification procedures, and troubleshooting guidelines for the Predictive Maintenance and Failure Risk Assessment for Industrial Equipment project. The system operationalizes an end-to-end machine learning pipeline as an accessible, web-based decision-support prototype hosted on Render.

The underlying machine learning architecture utilizes a tuned XGBoost classifier trained on the authoritative AI4I 2020 Predictive Maintenance benchmark dataset (10,000 operational records). The production pipeline is serialized as a scikit-learn Pipeline object combining StandardScaler and the tuned XGBoost model with cross-validated hyperparameters. Raw continuous telemetry measurements are passed directly into the pipeline during inference, guaranteeing identical preprocessing transformations without runtime data leakage.

The deployment serves both human operators and automated client systems through distinct interfaces:

- **Operator Dashboard (Web Form):** Interactive web dashboard featuring six sensor telemetry input fields, two-level validation, live risk gauges, advisory maintenance guidance, and empirical SHAP global feature impact rankings.
- **Client Integration (REST API):** Structured REST API endpoints (GET /api/metadata and POST /api/predict) providing programmatic model metadata and JSON-formatted probabilistic failure risk assessments.

2. Repository & Deployment Links

The project is publicly version-controlled on GitHub and deployed directly from the authoritative main branch to Render's cloud platform. All production endpoints and repositories are summarized below:

Resource / Service	Endpoint / Location Details	Operational Notes
Project Title	Predictive Maintenance and Failure Risk Assessment for Industrial Equipment	Official academic and deployment project branding
GitHub Repository	https://github.com/S-Dinesh-Reddy/predictive-maintenance-failure-risk-assessment	Public Git repository containing code, models, and docs
Production Branch	main	Authoritative release branch tracked by Render auto-deploy
Live Web Application	https://predictive-maintenance-failure-risk.onrender.com	Primary operator dashboard interface hosted on Render

Resource / Service	Endpoint / Location Details	Operational Notes
REST Metadata API	https://predictive-maintenance-failure-risk.onrender.com/api/metadata	Returns JSON model specifications, CV metrics, and threshold
REST Prediction API	https://predictive-maintenance-failure-risk.onrender.com/api/predict	Accepts JSON telemetry payload; returns 5 verified fields
Operational Threshold	0.50 (Retained Decision Boundary)	Prioritizes failure detection to minimize missed failures

3. Render Hosting & Service Configuration

The application is hosted as a managed Linux-based Python Web Service on Render. The service is connected directly to the GitHub repository via Render's GitHub App integration. On every commit pushed to the main branch, Render triggers an automated cloud build, compiles dependencies, and seamlessly updates the live container instance.

The table below specifies the exact technical parameters established in the Render service console:

Configuration Parameter	Configured Value	Technical Rationale & Verification
Service Type	Web Service	Provides managed HTTP/HTTPS routing, reverse proxy, and public SSL
Environment / Runtime	Python 3 (Python 3.11.x)	Matches local development runtime and precompiled ML package wheels
Repository Name	S-Dinesh-Reddy/predictive-maintenance-failure-risk-assessment	Connected via authenticated GitHub repository integration
Deployment Branch	main	Tracks authoritative tested commits; initiates automated builds
Root Directory	Blank (Repository Root)	Allows Render to resolve requirements.txt and app.py at top-level
Build Command	pip install -r requirements.txt	Installs Flask, scikit-learn, XGBoost, Gunicorn, and dependencies
Start Command	gunicorn app:app	Launches production WSGI server pointing to Flask instance in app.py
Instance Type	Free Tier (0.1 CPU, 512 MB RAM)	Sufficient compute and memory for lightweight prototype inference
Network Port	Dynamic $PORT (binds to 10000)	Render automatically injects $PORT; Gunicorn binds to 0.0.0.0:$PORT
Auto-Deploy	Yes (Enabled)	Continuous deployment triggered automatically upon git push to main

4. Step-by-Step Deployment Procedure

To reproduce or re-establish this deployment from scratch, follow these structured steps:

Step 1 — Verify Local Repository & Model Artifacts: Verify that requirements.txt includes gunicorn>=23.0.0 alongside flask, scikit-learn, pandas, numpy, and xgboost. Verify that the serialized production pipeline (models/final_pipeline.pkl) and model metadata (models/model_metadata.json) exist on disk and are tracked in Git. Execute tests/test_pipeline_and_app.py locally to confirm that all 5 test suites pass.

Step 2 — Authenticate with Render Cloud Platform: Log in to the Render management console (https://dashboard.render.com). Ensure GitHub account authorization is established with permissions granting access to the predictive-maintenance-failure-risk-assessment repository.

Step 3 — Create a New Web Service: In the Render dashboard, click the 'New +' button in the top navigation and select 'Web Service'. In the repository selection prompt, search for and choose S-Dinesh-Reddy/predictive-maintenance-failure-risk-assessment.

Step 4 — Define Deployment Specifications: Configure the core service parameters as follows: Name = predictive-maintenance-failure-risk, Region = Oregon (US West) or closest regional datacenter, Branch = main, Root Directory = leave completely blank, Runtime = Python 3.

Step 5 — Set Build & Start Commands: Under the 'Build Command' input, enter exactly: pip install -r requirements.txt. Under the 'Start Command' input, enter exactly: gunicorn app:app. These commands instruct Render to install required package wheels into an isolated virtual environment and start Gunicorn targeting the Flask app object.

Step 6 — Select Compute Tier and Launch: Select the 'Free' instance plan (0.1 CPU, 512 MB RAM). Click 'Create Web Service' (or 'Deploy Web Service') to initiate the initial container build and deployment process.

Step 7 — Monitor Build & Runtime Logs: Monitor the streaming build logs in the Render console. A successful deployment log stream will show package resolution, dependency installation, container packaging, Gunicorn process startup on port 10000, and a final notification: 'Your service is live □'.

Step 8 — Perform Post-Deployment Smoke Verification: Navigate to the primary public URL (https://predictive-maintenance-failure-risk.onrender.com). Confirm that the dashboard loads, input form controls are responsive, and test telemetry evaluates correctly.

5. Local Preparation & Git Workflow

When updating code, templates, or documentation, changes are committed locally and pushed to GitHub. When auto-deploy is enabled, Render detects the push event and automatically starts a new cloud deployment. The standard PowerShell command sequence is detailed below:

```
# 1. Verify working directory status and modified files
git status

# 2. Stage updated application code, documentation, and assets
git add app.py requirements.txt templates/ static/ docs/

# 3. Create a descriptive commit summarizing changes
git commit -m "Update predictive maintenance application and deployment documentation"

# 4. Push updates to GitHub main branch to trigger Render auto-deploy
git push origin main
```

If auto-deploy is temporarily paused or a rebuild with a refreshed package cache is required, navigate to the Render service dashboard, click 'Manual Deploy', and choose 'Clear build cache & deploy'.

6. Comprehensive Verification Checklist

A rigorous post-deployment verification was conducted across all application subsystems. The verification checklist below confirms the operational integrity of the live Render deployment:

Verification Item	Expected Operational Behavior	Verified Result & Evidence
Cloud Build Execution	Clean compilation and installation of all requirements.txt wheels	PASSED — Build completed with exit code 0; 0 missing wheels
WSGI Server Startup	Gunicorn boots master worker and binds to dynamic cloud port	PASSED — Gunicorn listening at 0.0.0.0:10000; service marked live
HTTPS Security & Routing	Encrypted SSL/TLS connection established with valid certificate	PASSED — HTTPS 200 OK verified on public onrender.com domain
Web Dashboard Delivery	Full industrial user interface renders with styles and input cards	PASSED — Dark-mode UI, fonts, machine diagram, and forms load
Metadata REST API	GET /api/metadata returns JSON model specs and CV metrics	PASSED — HTTP 200 returned with test F1: 0.8058, ROC-AUC: 0.9758
Form Telemetry Inference	POST /predict processes sensor form data and updates DOM	PASSED — Renders failure probability gauge, risk badge, and banner
Programmatic REST API	POST /api/predict accepts JSON payload and returns prediction	PASSED — Returns 5 verified fields (status, prediction, prob, risk, threshold)
Level 1 Sanity Limits	Rejects out-of-bound or negative inputs (e.g., Torque > 150 Nm)	PASSED — HTTP 400 with descriptive error message returned
Level 2 Range Warnings	Advisory amber banner when telemetry exceeds training range	PASSED — '⚠ Outside Model Training Range' alert displayed
Static Asset Integrity	CSS stylesheets, JavaScript scripts, and images load without error	PASSED — Zero 404 responses observed in network inspector
Environment Portability	Zero hardcoded local Windows file paths invoked during runtime	PASSED — Relative path resolution via os.path.dirname(__file__)

7. Troubleshooting & Common Deployment Issues

This section documents common deployment issues encountered on cloud platforms like Render, along with their root causes and concrete remediation steps:

Issue 1 — Build Error: 'requirements.txt not found': Confirm that the 'Root Directory' setting in Render is left completely blank. When the repository contains requirements.txt at the top level, leaving Root Directory blank ensures pip resolves the file in the current working directory. Do not enter build commands into the directory field.

Issue 2 — Startup Error: 'Gunicorn cannot import app:app': Verify that app.py is located at the repository root and that the Flask application instance is assigned to a variable named app (app = Flask(__name__)). If the file or variable is named differently (e.g., main.py or application), the Start Command must be updated accordingly (e.g., gunicorn main:app).

Issue 3 — Runtime Error: 'FileNotFoundError: models/final_pipeline.pkl': Ensure that models/final_pipeline.pkl and models/model_metadata.json are committed to Git. Check .gitignore to confirm that models/*.pkl is not unintentionally excluded. In app.py, always construct model paths using os.path.join(BASE_DIR, 'models', 'final_pipeline.pkl') rather than hardcoded local file paths.

Issue 4 — Initial Latency on First Visit ('Cold Start Delay'): On the Render Free tier, container instances automatically spin down ('sleep') after 15 minutes of inactivity to preserve shared cloud infrastructure. When a user visits the URL after an idle period, Render must allocate a container, pull the image, and start the process. This 'cold start' takes approximately 30 to 50 seconds. Subsequent requests are handled immediately with sub-second latency. This is standard free-tier platform behavior.

Issue 5 — Memory Limit Exceeded during Build (OOM Killer): If the build fails due to memory exhaustion, ensure requirements.txt contains only runtime dependencies rather than heavy development or notebook export packages (such as jupyterlab, torch, or tensorflow). The lightweight stack (Flask, scikit-learn, XGBoost, Gunicorn) requires less than 350 MB during compilation.

8. Limitations & Responsible Use Notes

In accordance with professional engineering ethics and academic reporting standards, the operational limitations of this cloud-deployed prototype must be clearly recognized:

- **Synthetic Simulation Origin:** The prototype is trained and benchmarked on the synthetic AI4I 2020 dataset (10,000 records). While the mathematical modeling accurately captures intended failure trade-offs, it represents an idealized simulation rather than real-world CNC milling machine tool degradation.
- **Advisory Risk Estimations:** Model outputs represent statistical risk estimations derived from sensor telemetry, not definitive physical guarantees of mechanical failure or remaining useful life (RUL).
- **Non-Safety-Critical Architecture:** The web application functions strictly as an operator advisory decision-support prototype. It does not interface with industrial programmable logic controllers (PLCs), physical safety relays, or emergency-stop (E-stop) hardware circuits.
- **Prerequisites for Industrial Transition:** Actual industrial equipment failure prediction relies heavily on high-frequency piezoelectric accelerometer vibration telemetry (10 kHz to 20 kHz sampling), acoustic emissions, and continuous time-series degradation tracking. Real-world deployment requires rigorous validation using operational shop-floor telemetry and human oversight by certified maintenance reliability engineers.

9. Deployment Status Summary

The production deployment of the Predictive Maintenance and Failure Risk Assessment system is verified, active, and fully operational on Render. The live cloud service accurately reflects the repository's main branch, adheres to verified academic specifications, and provides a dependable reference for project evaluation and submission.

Deployment Attribute	Verified Production Status	Details & Metrics
Deployment Health	ACTIVE / LIVE	Container healthy and serving incoming traffic
Public Cloud URL	https://predictive-maintenance-failure-risk.onrender.com	Verified responsive via HTTP/2 and TLS 1.3
Repository Synchronization	Synchronized with main branch	Tracks commit history on GitHub (S-Dinesh-Reddy)
Inference Pipeline	XGBoost + StandardScaler Pipeline	F1: 0.8058, Recall: 82.35%, Precision: 78.87%, ROC-AUC: 0.9758

Deployment Attribute	Verified Production Status	Details & Metrics
Decision Boundary	Threshold = 0.50	Optimized for failure sensitivity on imbalanced telemetry
Documentation Status	Complete & Standardized	Aligned with Project_Documentation.docx academic format